

Assignment 9

Step 1: Problem Statement

Create a virtual train represented by a linked list that will be populated in with railcar objects in "NY" and then begin a 4-stop journey to "Florida". The stops will be in the virtual cities of Washington D.C., Charleston, Orlando, and W. Palm Beach. At each stop the railcar objects with the corresponding city as their destination, will be removed from the linkedlist train. One small twist, in Orlando, the railcar objects containing parrots as their cargo must also be removed. Additionally, the progress of our cars/train must be displayed at key points during the journey and displayed as empty at the end.

Lastly, we must display an entire list of our railcar objects from a doubly linkedlist printed backwards.

Step 2: What I know

Creating the rail car objects should be easy enough.

LinkedLists both singly & doubly should be easy enough.

Step 2: (cont)

manipulating & displaying the lists shouldn't be an issue.

lots of experience reading from files to populate data structures.

Step 2: what I don't know

The custom sorting by destination may be a little challenging. Had some issues with it last time but hoping that experience will facilitate an easier outcome this time around.

Step 3: pseudo code

Main

Railcar

- number
- freight
- destination

Constructor: railcar(num, frt, dest)

getters: freight & Dest

Setters: none

To String()

Railcar Freight Destination City

xx

xx

2 - 3

Int compareTo (Railcar Other Rail car)

→

Step 3: (cont)

Not Compare To (railcar & other railcar)

Front		Back
West Palm	→ Orlando → Charleston →	DC
Priority: Low		→ High

Train Linked List

~ Head

~~Size~~

Constructor - Default

Getters/setters - none

Methods:

Add by Destination (railcar to add)

remove by Destination (destination)

remove by freight (String Freight)

displayTrain() (convert String in railcar)

~~While (linked list is empty) {~~

~~if (head.f Freight == freight) {~~

~~head = previous to head + 1~~

~~else {~~

~~if (head.element == freight) {~~

~~removeFirst~~

~~else if~~

~~node previous = head;~~

~~for (i = 0; i < linkedList.size; i++)~~

~~prev = prev.next;~~

~~node temp = prev.next;~~

~~prev.next = temp; let~~

~~size =~~

Step 3 (cont)

~~Remove~~ Remove by Freight.

Current = head

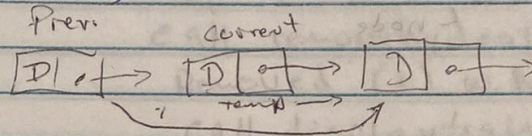
while (current != null) {

if (current.data == Freight) {

remove current

current = current.next

}



Remove current

if head.Freight == Freight

Remove First

node prev = ~~Current~~ next - 1

temp = prev.next

prev.next = temp.next

Private Inner

class Node

- railcar

- next

constructor (x)

- initialize railcar

- initialize next

Deebly Linked

Main

- Railcar

- Train Linkedlist

 - ↳ node

- Double Linkedlist

 - ↳ node

main

Read Files

call railcars to create objects

add by dest → call Linked List to add to train

add to end → call double linked list to add to list

Train depart New York

call display ~~train~~

stop 1 Train arrives in DC

call remove by destination (DC)

print removed (x) DC rail cars

call display train

stop 2 Train arrives in charl.

call remove by dest (charlston)

print removed (x) charl rail cars

call display train

stop 3 train arrives in orlando

Step 3 (cont)

call removeByDest(Orlando)
Print removed(x) Orlando rail cars
call removeByFreight(Parrots)
print removed(x) cars with Parrots
call displaytrain();

Step 4 Train arrives in W. Palm

call removeByDest(W. Palm);
Removed (x) W. Palm rail cars - ^{Train} should be empty
call displaytrain();

Rail cars in doubly linked list - Printed Back
Call displaybackwards();

Step 4 what did I learn

I Learned my fears were justified
I was unable to populate the
singly linked list with the
overridden compareTo method.
Hours of work and research
and still unsuccessful. turning
in the work I completed and
will try again on my own after
the submission as I'm out of
time.